



Guía de Estudio: Fundamentos de Python para tu Examen

Una guía práctica y enfocada en lo que realmente necesitas dominar.

1. SINTAXIS Y ESTRUCTURA DEL LENGUAJE

Qué debes saber

La sintaxis es la gramática de Python. Si no la dominas, nada funciona.

Conceptos clave

Concepto	Descripción	Ejemplo
Indentación	Python usa espacios (4) para bloques, no <code>{ }</code>	<pre>if True:\n print("indentado")</pre>
Variables	Nombres que guardan valores	<pre>nombre = "Ana"</pre>
Tipos dinámicos	Python infiere el tipo automáticamente	<pre>x = 5</pre> es int, <pre>x = "5"</pre> es str
Comentarios	<pre>#</pre> para comentarios de una línea, <pre>""" ... """</pre> para bloques	<pre># Esto es un comentario</pre>

Ejercicio práctico

```
python

# ✅ Correcto: Sintaxis válida
nombre = "Juan"
edad = 25
salario = 2500.50

# ❌ Incorrecto: Falta indentación
if nombre:
print("Hola") # Error: IndentationError

# ✅ Correcto:
if nombre:
    print("Hola") # Bien indentado
```

2. 📁 TIPOS DE DATOS

Los 5 tipos fundamentales que debes dominar

int (Entero)

```
python

edad = 25
cantidad = -10
numero = 0

# Operaciones
resultado = 10 + 5 # 15
potencia = 2 ** 3 # 8 (exponenciación)
division = 10 // 3 # 3 (división entera)
resto = 10 % 3 # 1 (módulo)
```

Cuándo usarlo: Conteos, índices, cálculos sin decimales

float (Decimal)

```
python

altura = 1.75
temperatura = 36.5
pi = 3.14159

# Operaciones
area = 3.14 * 5 ** 2 # 78.5

# Cuidado: Precisión
print(0.1 + 0.2) # 0.30000000000000004 (no exactamente 0.3)
```

Cuándo usarlo: Medidas reales, cálculos científicos, dinero (aunque mejor usar Decimal)

str (Texto)

```
python
```

```

nombre = "Python"
saludo = 'Hola'
mensaje = """Texto
multilínea"""

# Operaciones
concatenacion = "Hola" + " " + "mundo" # "Hola mundo"
repeticion = "la" * 3 # "lalala"
longitud = len("Python") # 6

# Indexación (empieza en 0)
letra = "Python"[0] # 'P'
ultima = "Python"[-1] # 'n' (desde el final)

# Slicing
primeros_3 = "Python"[0:3] # 'Pyt'
desde_indice_2 = "Python"[2:] # 'thon'

# Métodos útiles
minuscula = "PYTHON".lower() # 'python'
mayuscula = "python".upper() # 'PYTHON'
reemplazar = "Hola mundo".replace("mundo", "Python") # "Hola Python"
dividir = "a,b,c".split(",") # ['a', 'b', 'c']

```

Cuándo usarlo: Nombres, mensajes, datos de entrada del usuario

bool (Booleano)

```

python

verdadero = True
falso = False

# Operaciones lógicas
resultado = True and False # False
resultado = True or False  # True
resultado = not True        # False

# Comparaciones devuelven bool
es_mayor = 10 > 5 # True
es_igual = 10 == 5 # False
es_diferente = 10 != 5 # True

```

Cuándo usarlo: Condiciones, validaciones, banderas

type() - Saber el tipo

python

```
print(type(5))           # <class 'int'>
print(type(5.0))         # <class 'float'>
print(type("5"))         # <class 'str'>
print(type(True))        # <class 'bool'>
```

Conversión de tipos

python

```
# Convertir a int
numero = int("42")       # 42
numero = int(3.99)       # 3 (trunca)

# Convertir a float
decimal = float("3.14")  # 3.14
decimal = float(5)       # 5.0

# Convertir a str
texto = str(42)          # "42"
texto = str(3.14)        # "3.14"

# Convertir a bool
booleano = bool(1)       # True
booleano = bool(0)       # False
booleano = bool("")      # False (strings vacíos = False)
booleano = bool([])      # False (listas vacías = False)
```

3. 📦 LISTAS, TUPLAS, DICCIONARIOS Y CONJUNTOS

Estructuras de datos:Cuál usar cuándo

● LISTAS `[]` - Ordenadas, modificables

python

```
# Crear
numeros = [1, 2, 3, 4, 5]
mixta = [1, "texto", 3.14, True]
vacía = []

# Acceso
primer = numeros[0]      # 1
ultimo = numeros[-1]     # 5
sublista = numeros[1:4]  # [2, 3, 4]

# Modificar
numeros[0] = 10           # [10, 2, 3, 4, 5]
numeros.append(6)         # [10, 2, 3, 4, 5, 6]
numeros.insert(0, 0)      # [0, 10, 2, 3, 4, 5, 6]
numeros.remove(10)        # [0, 2, 3, 4, 5, 6] (por valor)
removido = numeros.pop()  # Quita el último, lo retorna
numeros.extend([7, 8])    # Agrega múltiples elementos

# Información
longitud = len(numeros)   # 8
existe = 5 in numeros     # True
índice = numeros.index(5) # Posición del 5

# Ordenar
numeros.sort()            # Ordena en lugar
copia_ordenada = sorted(numeros) # Retorna ordenada

# Invertir
numeros.reverse()         # Invierte en lugar
invertida = numeros[::-1] # Invierte sin modificar
```

Cuándo usarla: Cuando necesitas colecciones que cambian (agregar, quitar)

● TUPLAS `()` - Ordenadas, inmutables

python

```
# Crear
coordenadas = (10, 20)
punto_3d = (1, 2, 3)
una_sola = (5,) # ¡Necesita coma! Sino es solo (5)

# Acceso
x = coordenadas[0] # 10
y = coordenadas[-1] # 20

# NO se puede modificar
# coordenadas[0] = 15 # Error: TypeError

# Útil: Desempacar
x, y = (10, 20)
print(x, y) # 10 20

# Útil: Retornar múltiples valores
def obtener_coordenadas():
    return (10, 20)

x, y = obtener_coordenadas()
```

Cuándo usarla: Datos que no deben cambiar, coordenadas, claves de diccionario

● DICCIONARIOS `{}` - Clave-valor, modificables

python

```
# Crear
persona = {
    "nombre": "Juan",
    "edad": 25,
    "ciudad": "Madrid"
}

# Acceso
nombre = persona["nombre"] # "Juan"
edad = persona.get("edad") # 25 (seguro, retorna None si no existe)
edad_defecto = persona.get("telefono", "No tiene") # "No tiene"

# Modificar/Agregar
persona["edad"] = 26 # Modifica
persona["email"] = "juan@mail.com" # Agrega nueva clave

# Eliminar
del persona["ciudad"] # Elimina la clave
removido = persona.pop("email") # Quita y retorna el valor

# Información
claves = persona.keys() # ['nombre', 'edad', ...]
valores = persona.values() # ['Juan', 26, ...]
pares = persona.items() # [('nombre', 'Juan'), ('edad', 26), ...]

# Iterar
for clave, valor in persona.items():
    print(f"{clave}: {valor}")

# Búsqueda
existe = "nombre" in persona # True
```

Cuándo usarlo: Datos con propiedades (usuario, configuración), búsquedas rápidas

● CONJUNTOS `{}` - Sin orden, sin duplicados

python

```
# Crear
numeros = {1, 2, 3, 4, 5}
frutas = {"manzana", "banana", "naranja"}

# Duplicados se eliminan
numeros = {1, 1, 2, 2, 3} # {1, 2, 3}

# Operaciones
numeros.add(6)           # Agrega
numeros.remove(1)        # Quita (error si no existe)
numeros.discard(100)     # Quita (sin error)
numeros.clear()          # Vacía el conjunto

# Operaciones matemáticas
A = {1, 2, 3}
B = {3, 4, 5}

union = A | B             # {1, 2, 3, 4, 5}
interseccion = A & B      # {3}
diferencia = A - B        # {1, 2}
simetrica = A ^ B         # {1, 2, 4, 5}

# Búsqueda rápida
esta = 3 in numeros       # True
```

Cuándo usarlo: Colecciones únicas, operaciones matemáticas de conjuntos

 Resumen de Colecciones

	Lista	Tupla	Diccionario	Conjunto
Sintaxis	[1, 2]	(1, 2)	{"a": 1}	{1, 2}
Ordenado	✓	✓	✓ (3.7+)	✗
Modificable	✓	✗	✓	✓
Índices	✓	✓	✗	✗
Duplicados	✓	✓	✗ (claves)	✗
Caso de uso	Arrays dinámicos	Datos inmutables	Mapeos clave-valor	Colecciones únicas

4. CONTROL DE FLUJO

Decisiones y repeticiones

Condicionales: if, elif, else

```
python
```

```
edad = 18

# if simple
if edad >= 18:
    print("Eres mayor de edad")

# if-else
if edad >= 18:
    print("Mayor de edad")
else:
    print("Menor de edad")

# if-elif-else
if edad < 13:
    print("Niño")
elif edad < 18:
    print("Adolescente")
elif edad < 65:
    print("Adulto")
else:
    print("Jubilado")

# Operadores de comparación
if edad == 18: # Igual
    print("Tienes 18")

if edad != 18: # Diferente
    print("No tienes 18")

if edad > 18: # Mayor
    print("Más de 18")

if edad >= 18: # Mayor o igual
    print("18 o más")

if edad < 18: # Menor
    print("Menos de 18")

if edad <= 18: # Menor o igual
    print("18 o menos")

# Operadores lógicos
if edad >= 18 and nombre == "Juan":
    print("Juan mayor de edad")

if edad < 13 or edad > 65:
```

```
    print("Categoría especial")

if not es_admin:
    print("No es administrador")

# Condición en una línea
estado = "Adulto" if edad >= 18 else "Menor"
print(estado)
```

Bucles: for

python

```
# For sobre lista
frutas = ["manzana", "banana", "naranja"]
for fruta in frutas:
    print(fruta)

# For con índice
for i in range(len(frutas)):
    print(f"{i}: {frutas[i]}")

# For con enumerate (índice + valor)
for indice, fruta in enumerate(frutas):
    print(f"{indice}: {fruta}")

# For sobre diccionario
persona = {"nombre": "Juan", "edad": 25}
for clave, valor in persona.items():
    print(f"{clave}: {valor}")

# For sobre rango
for i in range(5):          # 0, 1, 2, 3, 4
    print(i)

for i in range(1, 6):      # 1, 2, 3, 4, 5 (hasta 6 sin incluir)
    print(i)

for i in range(0, 10, 2):  # 0, 2, 4, 6, 8 (de 2 en 2)
    print(i)

# For con break (salir)
for i in range(10):
    if i == 5:
        break # Sale del bucle
    print(i) # Imprime 0, 1, 2, 3, 4

# For con continue (siguiente iteración)
for i in range(10):
    if i == 5:
        continue # Salta a la siguiente iteración
    print(i) # Imprime 0, 1, 2, 3, 4, 6, 7, 8, 9
```

Bucles: while

python

```
contador = 0
while contador < 5:
    print(contador)
    contador += 1 # Equivalente a: contador = contador + 1

# While con break
numero = 1
while True:
    print(numero)
    if numero == 5:
        break
    numero += 1

# While con continue
numero = 0
while numero < 10:
    numero += 1
    if numero % 2 == 0: # Si es par
        continue
    print(numero) # Imprime los impares
```

Iteración sobre colecciones

python

```
# Sobre listas
lista = [1, 2, 3, 4, 5]
for elemento in lista:
    print(elemento)

# Sobre tuplas
tupla = (10, 20, 30)
for elemento in tupla:
    print(elemento)

# Sobre diccionarios
diccionario = {"a": 1, "b": 2, "c": 3}
for clave in diccionario: # Solo claves
    print(clave)

for clave, valor in diccionario.items(): # Clave y valor
    print(f"{clave}: {valor}")

# Sobre strings
texto = "Python"
for letra in texto:
    print(letra)
```

5. FUNCIONES Y MODULARIDAD

El arte de reutilizar código

Definición básica

python

```
# Función sin parámetros ni retorno
def saludar():
    print("¡Hola!")

saludar() # Llamada

# Función con parámetros
def saludar_a(nombre):
    print(f"¡Hola, {nombre}!")

saludar_a("Juan") # ¡Hola, Juan!

# Función con retorno
def sumar(a, b):
    resultado = a + b
    return resultado

total = sumar(5, 3)
print(total) # 8

# Función con múltiples retornos
def obtener_datos():
    return "Juan", 25, "Madrid"

nombre, edad, ciudad = obtener_datos()
```

Parámetros y argumentos

python

```
# Parámetros posicionales (importa el orden)
def presentarse(nombre, edad, ciudad):
    print(f"{nombre} tiene {edad} años y vive en {ciudad}")

presentarse("Juan", 25, "Madrid") # Correcto
# presentarse("Madrid", "Juan", 25) # Error: tipos incorrectos

# Parámetros con nombre (no importa el orden)
presentarse(edad=25, ciudad="Madrid", nombre="Juan") # Funciona

# Mezcla
presentarse("Juan", edad=25, ciudad="Madrid") # También funciona

# Parámetros por defecto
def saludar(nombre, saludo="Hola"):
    print(f"{saludo}, {nombre}")

saludar("Juan") # Hola, Juan
saludar("Juan", "Buenos días") # Buenos días, Juan

# *args: número variable de argumentos (como tupla)
def sumar_varios(*numeros):
    total = 0
    for numero in numeros:
        total += numero
    return total

resultado = sumar_varios(1, 2, 3, 4, 5) # 15

# **kwargs: argumentos con nombre variable (como diccionario)
def crear_usuario(**datos):
    for clave, valor in datos.items():
        print(f"{clave}: {valor}")

crear_usuario(nombre="Juan", edad=25, email="juan@mail.com")

# Combinado
def funcion_completa(a, b, *args, **kwargs):
    print(f"a={a}, b={b}")
    print(f"args={args}")
    print(f"kwargs={kwargs}")

funcion_completa(1, 2, 3, 4, nombre="Juan", edad=25)
# a=1, b=2
```



```
# args=(3, 4)
# kwargs={'nombre': 'Juan', 'edad': 25}
```

Funciones lambda (Funciones anónimas)

```
python

# Sintaxis: lambda argumentos: expresión

# Función normal
def sumar(a, b):
    return a + b

# Equivalente con lambda
sumar_lambda = lambda a, b: a + b
print(sumar_lambda(5, 3)) # 8

# Uso común: con map, filter, sorted
numeros = [1, 2, 3, 4, 5]

# Doblar cada número
duplicados = list(map(lambda x: x * 2, numeros))
print(duplicados) # [2, 4, 6, 8, 10]

# Filtrar pares
pares = list(filter(lambda x: x % 2 == 0, numeros))
print(pares) # [2, 4]

# Ordenar diccionarios
personas = [
    {"nombre": "Juan", "edad": 25},
    {"nombre": "Ana", "edad": 20},
    {"nombre": "Carlos", "edad": 30}
]

por_edad = sorted(personas, key=lambda p: p["edad"])
# [{'nombre': 'Ana', 'edad': 20}, ...]
```

Alcance de variables (scope)

```
python
```

```
# Global vs Local
x = 10 # Global

def funcion():
    x = 5 # Local (sombreada la global)
    print(x) # 5

funcion()
print(x) # 10 (la global no cambi6)

# Acceder a global con 'global'
contador = 0

def incrementar():
    global contador
    contador += 1
    print(contador)

incrementar() # 1
incrementar() # 2

# Regla LEGB (Local, Enclosing, Global, Built-in)
def externa():
    y = 20 # Enclosing

    def interna():
        z = 5 # Local
        print(z) # 5 (local)
        print(y) # 20 (enclosing)

    interna()

interna() # Error: interna no existe aqu 
externa() # Funciona
```

Importaci6n de m6dulos

python

```
# Importar módulo completo
import math
print(math.sqrt(16)) # 4.0
print(math.pi)      # 3.14159...

# Importar función específica
from math import sqrt, pi
print(sqrt(16))      # 4.0
print(pi)            # 3.14159...

# Importar con alias
import math as m
print(m.sqrt(16))    # 4.0

from math import sqrt as raiz_cuadrada
print(raiz_cuadrada(16)) # 4.0

# Importar todo (no recomendado)
from math import *
print(sqrt(16))      # Funciona pero es confuso

# Tu propio módulo
# archivo: mi_modulo.py
# def saludar(nombre):
#     return f"Hola, {nombre}"

# En otro archivo:
# from mi_modulo import saludar
# print(saludar("Juan"))
```

6. 🏠 PROGRAMACIÓN ORIENTADA A OBJETOS (POO)

Pensar en objetos, no solo funciones

Clases y objetos

```
python
```

```
# Definir una clase
class Persona:
    pass # Clase vacía

# Crear un objeto (instancia)
juan = Persona()
ana = Persona()

# Agregar atributos manualmente
juan.nombre = "Juan"
juan.edad = 25
ana.nombre = "Ana"
ana.edad = 20
```

Constructores y atributos

python

```
class Persona:
    # Constructor: se ejecuta al crear la instancia
    def __init__(self, nombre, edad):
        self.nombre = nombre # Atributo de instancia
        self.edad = edad
        self.experiencia = 0 # Valor por defecto

    # Método
    def presentarse(self):
        return f"Soy {self.nombre} y tengo {self.edad} años"

    def aumentar_experiencia(self, años):
        self.experiencia += años

# Crear instancias
juan = Persona("Juan", 25)
ana = Persona("Ana", 20)

# Acceder a atributos
print(juan.nombre) # Juan
print(juan.edad)   # 25

# Llamar métodos
print(juan.presentarse()) # Soy Juan y tengo 25 años

juan.aumentar_experiencia(3)
print(juan.experiencia) # 3
```

Métodos especiales

python

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def __str__(self):
        # Cómo se ve al imprimirlo
        return f"{self.nombre} ({self.edad} años)"

    def __repr__(self):
        # Representación técnica
        return f"Persona('{self.nombre}', {self.edad})"

    def __eq__(self, otro):
        # Comparación ==
        return self.nombre == otro.nombre and self.edad == otro.edad

    def __lt__(self, otro):
        # Comparación < (menor que)
        return self.edad < otro.edad

juan = Persona("Juan", 25)
ana = Persona("Ana", 20)

print(juan) # Juan (25 años)
print(repr(juan)) # Persona('Juan', 25)

print(juan == ana) # False
print(ana < juan) # True (Ana es menor)

# Ordenar por edad
personas = [juan, ana]
personas_ordenadas = sorted(personas) # Usa __lt__
```

Herencia simple

```
python
```

```
# Clase padre
class Animal:
    def __init__(self, nombre):
        self.nombre = nombre

    def hacer_sonido(self):
        return "Sonido genérico"

# Clase hija (hereda de Animal)
class Perro(Animal):
    def hacer_sonido(self): # Sobrescribe el método
        return "Guau guau"

class Gato(Animal):
    def hacer_sonido(self):
        return "Miau miau"

# Usar
perro = Perro("Rex")
gato = Gato("Misu")

print(perro.nombre) # Rex
print(perro.hacer_sonido()) # Guau guau

print(gato.nombre) # Misu
print(gato.hacer_sonido()) # Miau miau

# Verificar herencia
print(isinstance(perro, Animal)) # True
print(isinstance(perro, Perro)) # True
```

Encapsulamiento básico

```
python
```

```
class CuentaBancaria:
    def __init__(self, titular, saldo_inicial=0):
        self.titular = titular
        self.__saldo = saldo_inicial # Privado (__)

    def depositar(self, monto):
        if monto > 0:
            self.__saldo += monto
            return f"Depósito exitoso. Saldo: {self.__saldo}"
        return "Monto inválido"

    def retirar(self, monto):
        if 0 < monto <= self.__saldo:
            self.__saldo -= monto
            return f"Retiro exitoso. Saldo: {self.__saldo}"
        return "Monto inválido o saldo insuficiente"

    def obtener_saldo(self):
        return self.__saldo

cuenta = CuentaBancaria("Juan", 1000)
print(cuenta.depositar(500)) # Depósito exitoso. Saldo: 1500
print(cuenta.obtener_saldo()) # 1500
print(cuenta.retirar(200)) # Retiro exitoso. Saldo: 1300

# No se puede acceder directamente
# print(cuenta.__saldo) # Error: AttributeError
```

7. PROGRAMACIÓN CIENTÍFICA CON NUMPY Y VISUALIZACIÓN

Cálculos rápidos y gráficas

NumPy: Arreglos y operaciones vectorizadas

python


```
import numpy as np

# Crear arreglos
arreglo = np.array([1, 2, 3, 4, 5])
print(arreglo) # [1 2 3 4 5]

# Matriz (2D)
matriz = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

# Arreglo de ceros
ceros = np.zeros((3, 3))

# Arreglo de unos
unos = np.ones((2, 4))

# Rango
rango = np.arange(0, 10, 2) # [0, 2, 4, 6, 8]

# Linspace (espaciado uniforme)
lineal = np.linspace(0, 10, 5) # [0, 2.5, 5, 7.5, 10]

# Random
aleatorios = np.random.rand(3, 3) # 3x3 con valores entre 0 y 1
```

Indexación y slicing

python

```
arr = np.array([10, 20, 30, 40, 50])

# Indexación
primer = arr[0]      # 10
ultimo = arr[-1]     # 50

# Slicing
primeros_3 = arr[:3] # [10, 20, 30]
desde_2 = arr[2:]    # [30, 40, 50]
con_paso = arr[::2]   # [10, 30, 50] (cada 2)

# Matriz
matriz = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

elemento = matriz[1, 2] # 6 (fila 1, columna 2)
fila = matriz[0]        # [1, 2, 3]
columna = matriz[:, 1]   # [2, 5, 8]
submatriz = matriz[0:2, 1:3] # [[2, 3], [5, 6]]
```

Operaciones matemáticas

python

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
b = np.array([5, 4, 3, 2, 1])

# Operaciones básicas
suma = a + b          # [6, 6, 6, 6, 6]
resta = a - b         # [-4, -2, 0, 2, 4]
multiplicacion = a * b # [5, 8, 9, 8, 5]
division = a / b      # [0.2, 0.5, 1, 2, 5]
potencia = a ** 2     # [1, 4, 9, 16, 25]

# Funciones matemáticas
raiz = np.sqrt(a)     # [1, 1.41..., 1.73..., 2, 2.24...]
logaritmo = np.log(a) # [0, 0.69..., 1.09..., 1.38..., 1.60...]
exponencial = np.exp(a) # [2.71..., 7.38..., 20.08..., ...]

seno = np.sin(a)
coseno = np.cos(a)

# Estadísticas
promedio = np.mean(a) # 3.0
mediana = np.median(a) # 3.0
desviacion = np.std(a) # 1.41...
suma_total = np.sum(a) # 15
maximo = np.max(a)     # 5
minimo = np.min(a)     # 1

# Reshape (cambiar forma)
arr = np.arange(12) # [0, 1, 2, ..., 11]
matriz = arr.reshape(3, 4) # Matriz 3x4
vector = matriz.reshape(12) # Volver a vector
```

Generación de gráficas con Matplotlib

python

```
import matplotlib.pyplot as plt
import numpy as np

# Gráfica de línea
x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])

plt.figure(figsize=(8, 5))
plt.plot(x, y, marker='o', label='Datos')
plt.xlabel('Eje X')
plt.ylabel('Eje Y')
plt.title('Gráfica de Línea')
plt.legend()
plt.grid(True)
plt.show()

# Gráfica de barras
categorias = ['A', 'B', 'C', 'D']
valores = [10, 24, 36, 18]

plt.bar(categorias, valores, color='blue')
plt.xlabel('Categorías')
plt.ylabel('Valores')
plt.title('Gráfica de Barras')
plt.show()

# Gráfica de pastel
plt.pie(valores, labels=categorias, autopct='%1.1f%%')
plt.title('Gráfica de Pastel')
plt.show()

# Gráfica de dispersión
x_scatter = np.random.rand(50) * 100
y_scatter = np.random.rand(50) * 100

plt.scatter(x_scatter, y_scatter, alpha=0.6)
plt.xlabel('Variable X')
plt.ylabel('Variable Y')
plt.title('Gráfica de Dispersión')
plt.show()

# Histograma
datos = np.random.normal(100, 15, 1000)

plt.hist(datos, bins=30, edgecolor='black')
plt.xlabel('Valores')
```

```
plt.ylabel('Frecuencia')
plt.title('Histograma')
plt.show()

# Múltiples gráficas en una figura
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Subgráfica 1: Línea
axes[0, 0].plot(x, y)
axes[0, 0].set_title('Línea')

# Subgráfica 2: Barras
axes[0, 1].bar(categorías, valores)
axes[0, 1].set_title('Barras')

# Subgráfica 3: Dispersión
axes[1, 0].scatter(x_scatter, y_scatter)
axes[1, 0].set_title('Dispersión')

# Subgráfica 4: Histograma
axes[1, 1].hist(datos, bins=30)
axes[1, 1].set_title('Histograma')

plt.tight_layout()
plt.show()
```

Módulo Math (operaciones simples)

```
python
```

```
import math

# Constantes
pi = math.pi          # 3.14159...
e = math.e             # 2.71828...

# Funciones
raiz = math.sqrt(16)   # 4.0
potencia = math.pow(2, 3) # 8.0
factorial = math.factorial(5) # 120

# Trigonometría (en radianes)
seno = math.sin(math.pi / 2) # 1.0
coseno = math.cos(0)         # 1.0
tangente = math.tan(math.pi / 4) # 1.0

# Logaritmos
log_10 = math.log10(100) # 2.0
log_e = math.log(math.e) # 1.0

# Redondeamiento
ceil = math.ceil(3.2) # 4
floor = math.floor(3.8) # 3

# Valor absoluto
abs_val = abs(-5) # 5
```

8. 📁 MANEJO DE ARCHIVOS Y DATOS

Lectura, escritura y procesamiento

Manejo básico de archivos

```
python
```

```
# ESCRIBIR en archivo
with open("datos.txt", "w") as archivo:
    archivo.write("Línea 1\n")
    archivo.write("Línea 2\n")
    archivo.write("Línea 3\n")

# LEER archivo completo
with open("datos.txt", "r") as archivo:
    contenido = archivo.read()
    print(contenido)

# LEER línea por línea
with open("datos.txt", "r") as archivo:
    for linea in archivo:
        print(linea.strip()) # strip() quita \n

# LEER como lista de líneas
with open("datos.txt", "r") as archivo:
    lineas = archivo.readlines()
    print(lineas[0]) # Primera línea

# MODOS DE APERTURA
# "r" - Lectura (por defecto)
# "w" - Escritura (sobrescribe)
# "a" - Agregar (append)
# "x" - Crear (error si existe)
# "b" - Binario (fotos, videos, etc.)

# Ejemplo: Agregar al final
with open("datos.txt", "a") as archivo:
    archivo.write("Línea 4\n")
```

Series y DataFrames con Pandas

python

```
import pandas as pd

# SERIES (como una columna)
edades = pd.Series([25, 30, 20, 35])
print(edades)
# 0    25
# 1    30
# 2    20
# 3    35

# Series con índice personalizado
edades = pd.Series([25, 30, 20, 35], index=['Juan', 'Ana', 'Carlos', 'María'])
print(edades['Juan']) # 25

# DATAFRAME (como una tabla)
datos = {
    'nombre': ['Juan', 'Ana', 'Carlos'],
    'edad': [25, 30, 20],
    'ciudad': ['Madrid', 'Barcelona', 'Valencia']
}

df = pd.DataFrame(datos)
print(df)
#   nombre  edad  ciudad
# 0   Juan    25   Madrid
# 1   Ana     30 Barcelona
# 2  Carlos    20   Valencia

# Acceder a columnas
nombres = df['nombre']          # Series
edad_juan = df.loc[0, 'edad']   # Valor específico

# Primeras filas
print(df.head(2))               # Primeras 2 filas
print(df.tail(1))               # Última fila

# Información
print(df.info())                 # Tipos y no-nulos
print(df.describe())             # Estadísticas

# Operaciones
df['edad_proxima_ano'] = df['edad'] + 1
df_filtrado = df[df['edad'] > 25] # Filtrar por edad
```



```
# Ordenar
df_ordenado = df.sort_values('edad')
```

Lectura de CSV

```
python

import pandas as pd

# Leer CSV
df = pd.read_csv('datos.csv')

# Con opciones
df = pd.read_csv('datos.csv',
                  sep=';',          # Separador (por defecto ',')
                  encoding='utf-8', # Codificación
                  nrows=10)         # Solo primeras 10 filas

# Escribir CSV
df.to_csv('salida.csv', index=False)

# Ver primeras filas
print(df.head())

# Estadísticas
print(df.describe())

# Información de columnas
print(df.columns)
print(df.shape)  # (filas, columnas)
```

Lectura de Excel

```
python
```

```
import pandas as pd

# Leer Excel
df = pd.read_excel('datos.xlsx')

# Leer hoja específica
df = pd.read_excel('datos.xlsx', sheet_name='Sheet2')

# 0 por índice
df = pd.read_excel('datos.xlsx', sheet_name=0) # Primera hoja

# Escribir Excel
df.to_excel('salida.xlsx', index=False)

# Múltiples hojas
with pd.ExcelWriter('salida.xlsx') as writer:
    df1.to_excel(writer, sheet_name='Datos1')
    df2.to_excel(writer, sheet_name='Datos2')
```

Lectura de JSON

python

```
import json
import pandas as pd

# Leer JSON como diccionario
with open('datos.json', 'r') as archivo:
    datos = json.load(archivo)

print(datos)

# Escribir JSON
datos = {'nombre': 'Juan', 'edad': 25}
with open('salida.json', 'w') as archivo:
    json.dump(datos, archivo, indent=2)

# Con Pandas
df = pd.read_json('datos.json')
df.to_json('salida.json', orient='records')

# JSON en una línea (string)
json_string = '{"nombre": "Juan", "edad": 25}'
datos = json.loads(json_string)
print(datos['nombre']) # Juan

# String a JSON
json_string = json.dumps(datos)
print(json_string) # {"nombre": "Juan", "edad": 25}
```

9. ⚠ MANEJO DE ERRORES (try/except)

Prevenir que el programa se cuelgue

```
python
```

```
# Estructura básica
try:
    # Código que podría fallar
    numero = int("abc") # Error: "abc" no es un número
except ValueError:
    # Se ejecuta si ocurre ValueError
    print("Error: Debes ingresar un número válido")

# Capturar múltiples excepciones
try:
    lista = [1, 2, 3]
    numero = lista[10] # IndexError
except IndexError:
    print("Error: Índice fuera de rango")
except TypeError:
    print("Error: Tipo de dato incorrecto")

# Capturar cualquier excepción (no recomendado)
try:
    resultado = 10 / 0
except Exception:
    print("Ocurrió un error")

# else: se ejecuta si NO hay error
try:
    numero = int("123")
    print(f"Número: {numero}")
except ValueError:
    print("Error: No es un número válido")
else:
    print("Conversión exitosa")

# finally: SIEMPRE se ejecuta
try:
    archivo = open("datos.txt", "r")
    contenido = archivo.read()
except FileNotFoundError:
    print("Archivo no encontrado")
finally:
    archivo.close() # Se cierra aunque haya error

# Mejor práctica: usar 'with'
try:
    with open("datos.txt", "r") as archivo:
        contenido = archivo.read()
except FileNotFoundError:
```

```
    print("Archivo no encontrado")
# Archivo se cierra automáticamente

# Lanzar excepciones propias
def validar_edad(edad):
    if edad < 0 or edad > 150:
        raise ValueError("La edad debe estar entre 0 y 150")
    return True

try:
    validar_edad(200)
except ValueError as e:
    print(f"Error: {e}") # Error: La edad debe estar entre 0 y 150

# Excepciones comunes
# ValueError: Tipo correcto pero valor inválido
# TypeError: Tipo de dato incorrecto
# IndexError: Índice fuera de rango
# KeyError: Clave no existe en diccionario
# FileNotFoundError: Archivo no encontrado
# ZeroDivisionError: División por cero
# AttributeError: Atributo no existe
```

EJERCICIOS PRÁCTICOS POR TEMA

Para practicar antes del examen

1. Sintaxis y tipos de datos

- Crear variables de cada tipo
- Convertir entre tipos
- Realizar operaciones con cada tipo

2. Colecciones

- Crear lista, tupla, diccionario y conjunto
- Agregar/quitar elementos
- Buscar y acceder a elementos
- Iterar sobre cada una

3. Control de flujo

- Escribir programa con múltiples if/elif/else

- Hacer tabla de multiplicar con `for`
- Buscar elemento en lista con `while`
- Usar `break` y `continue`

4. Funciones

- Función que reciba 3 parámetros y retorne algo
- Función con parámetros por defecto
- Usar `lambda` con `map/filter`
- Hacer módulo con 2-3 funciones

5. POO

- Crear clase Estudiante con **`init`**, métodos
- Crear clase Profesor que herede de Persona
- Usar métodos especiales como **`str`**

6. NumPy y gráficas

- Crear array y hacer operaciones
- Hacer 4 gráficas diferentes
- Calcular estadísticas de array

7. Archivos y datos

- Leer CSV y mostrar primeras filas
- Escribir datos en JSON
- Usar `try/except` al abrir archivo

CONSEJOS PARA EL EXAMEN

1. **Lee bien las preguntas:** No assumes nada
2. **Sé específico:** `print()` retorna `None`, no muestra valores
3. **Cuidado con indentación:** Es fácil equivocarse
4. **Prueba ejemplos pequeños:** Antes de código complejo
5. **Maneja excepciones:** Cuando leas archivos o conviertas tipos
6. **Usa nombres claros:** `x`, `y` está bien en matemática, usa `edad`, `nombre` en general
7. **Comenta código complejo:** El comentario salva puntos
8. **Verifica tipos:** `type()` es tu amigo

Referencia Rápida

Imprescindibles:

- `len()`, `range()`, `print()`
- `append()`, `remove()`, `sort()`
- `keys()`, `values()`, `items()`
- `import`, `def`, `class`
- `if`, `for`, `while`, `try/except`

Frecuentes:

- `enumerate()`, `zip()`
- `list()`, `dict()`, `set()`, `tuple()`
- `str.split()`, `str.replace()`, `str.format()`
- `min()`, `max()`, `sum()`, `sorted()`

¡Buena suerte en tu examen! 📖 ✨

Recuerda: La práctica es clave. Haz ejercicios, comete errores, entiende por qué fallaron, y mejora.